

Ex.no:8	MINI PROJECT – HEALTHCARE ANALYTICS DASHBOARD

Domain and Problem Statement:

Domain: Healthcare Analytics

Problem Statement: Healthcare organizations need better insights into patient records, medical conditions, treatment patterns, and hospital performance. This project develops an interactive dashboard to analyze healthcare data and support data-driven decisions.

Dataset Description:

Dataset Name: Healthcare Patient Dataset

Granularity: Each row represents a patient record.

Key Entities & Attributes: Patient ID, Age, Gender, Diagnosis, Treatment, Hospital, Admission Date, Billing Amount and Medical Condition.

Tools Used

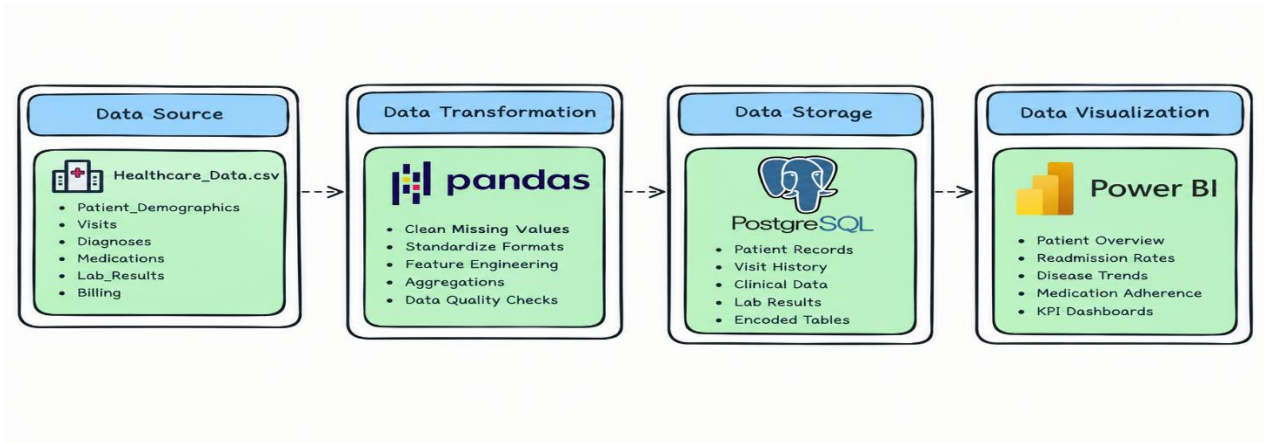
Python, Pandas, PostgreSQL, Power BI and Tableau.

Data Source:

Primary Source: Kaggle Open Datasets (Healthcare analytics Dataset).

Storage Infrastructure: Cleaned transactional records hosted within a local SQL database instance (healthcare_analytics).

Architecture:



Program:

1. Python – Data Cleaning & Transformation

```
import pandas as pd

from sqlalchemy import create_engine, text

# Load healthcare dataset
df = pd.read_csv("healthcare_data.csv")
healthcare = df.copy()

# Remove unnecessary columns
# Modify these columns according to your dataset
healthcare.drop(
    columns=["doctor_name", "hospital_address"],
    errors="ignore",
    inplace=True
)

# Handle missing values
healthcare["age"] = healthcare["age"].fillna(
    healthcare["age"].median()
)

healthcare["gender"] = healthcare["gender"].fillna("Unknown")
healthcare["diagnosis"] = healthcare["diagnosis"].fillna("Unknown")
healthcare["medication"] = healthcare["medication"].fillna("Unknown")

# Remove duplicate patient/visit records
healthcare.drop_duplicates(inplace=True)

healthcare["admission_date"] = pd.to_datetime(
    healthcare["admission_date"],
    errors="coerce"
)

healthcare["discharge_date"] = pd.to_datetime(
    healthcare["discharge_date"],
    errors="coerce"
)

healthcare["length_of_stay"] = (
    healthcare["discharge_date"] -
    healthcare["admission_date"]
```

```
).dt.days
# Create age groups
healthcare["age_group"] = pd.cut(
    healthcare["age"],
    bins=[0, 18, 35, 50, 65, 100],
    labels=[
        "0-18",
        "19-35",
        "36-50",
        "51-65",
        "65+"
    ]
)
if "bmi" in healthcare.columns:
    healthcare["bmi_category"] = pd.cut(
        healthcare["bmi"],
        bins=[0, 18.5, 25, 30, 100],
        labels=[
            "Underweight",
            "Normal",
            "Overweight",
            "Obese"
        ]
    )
if "billing_amount" in healthcare.columns:
    healthcare["billing_category"] = pd.cut(
        healthcare["billing_amount"],
        bins=[0, 10000, 50000, 100000, float("inf")],
        labels=[
            "Low",
            "Medium",
            "High",
            "Very High"
        ]
    )
```

```

# Filter invalid records
# Remove records with invalid age
healthcare = healthcare[
    (healthcare["age"] >= 0) &
    (healthcare["age"] <= 100)
]
healthcare = healthcare[
    healthcare["length_of_stay"].fillna(0) >= 0
]
if "billing_amount" in healthcare.columns:
    healthcare = healthcare[
        healthcare["billing_amount"].fillna(0) >= 0
    ]
healthcare.columns = (
    healthcare.columns
    .str.lower()
    .str.strip()
    .str.replace(" ", "_")
)
print("Original Shape:", df.shape)
print("Cleaned Shape:", healthcare.shape)
print("\nMissing Values:")
print(healthcare.isnull().sum())
print("\nCleaned Dataset:")
print(healthcare.head())

```

2. MySQL – Database and Data Loading

net start MySQL80

mysql -u root -p

Create Healthcare Analytics Database

```
CREATE DATABASE IF NOT EXISTS healthcare_analytics;
```

```
USE healthcare_analytics;
```

```
name = "root"
```

```
password = "YOUR_MYSQL_PASSWORD"
```

```
host = "localhost"
```

```
    f"mysql+pymysql://{username}:{password}@{host}:{port}/hotel_analytics"  
)
```

```
# Load cleaned data into MySQL
```

```
hotel.to_sql(  
    "hotel_bookings",  
    con=db_engine,  
    if_exists="replace",  
    index=False,  
    chunksize=5000  
)
```

```
print("Data loaded successfully into MySQL.")
```

Verify:

```
pd.read_sql(  
    "SELECT COUNT(*) AS total_rows FROM hotel_bookings",  
    db_engine  
)
```

Procedure:

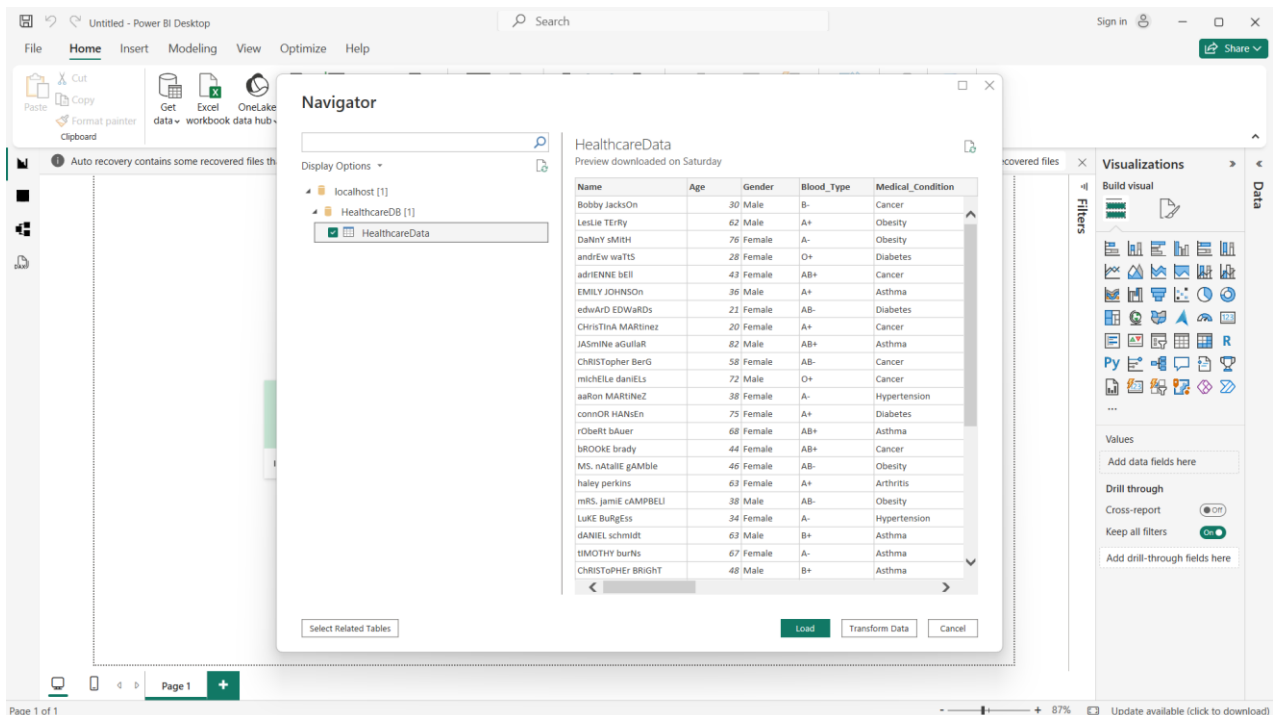
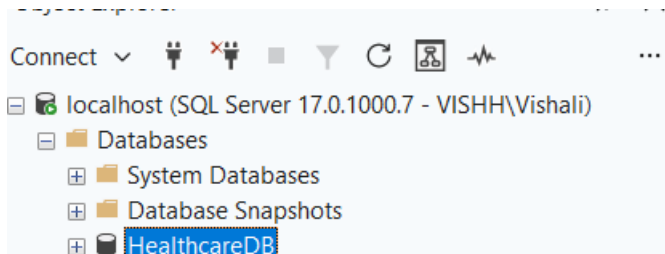
Step 1: Data Preparation & Preprocessing (Python / Pandas)

- Load the raw healthcare dataset into a Jupyter Notebook or Python script using `pandas.read_csv()`.
- Inspect the dataset for null values and handle missing data in important healthcare fields such as:
 - Age
 - Gender
 - Diagnosis
 - Medication
 - Clinical measurements
- Remove duplicate patient or healthcare visit records.

- Convert admission and discharge dates into proper datetime format.
- Feature engineer **length of hospital stay**
- Filter invalid records such as: Negative age, Age greater than 100, Negative hospital stay
- Standardize column names by converting them to lowercase and replacing spaces with underscores.
- Perform final data-quality checks and export the cleaned DataFrame to MySQL using SQLAlchemy and to_sql.

Step 2: SQL Database Connection & Data Ingestion

1. Open Power BI Desktop → Click **Get Data** → Select **MySQL Database**.
2. Enter the **Server** (localhost:3306) and **Database Name** (healthcare_analytics).
3. Enter authentication credentials, select the cleaned flat table (healthcare_analytics), and click **Transform Data** to open Power Query



Step 3: (Power BI) – Healthcare_Analytics

After loading the cleaned data into MySQL, connect **Power BI** to the `healthcare_analytics` database.

Patient Overview

- Total Patients
- Male vs Female Patients
- Average Patient Age
- Patient Distribution by Age Group

Hospital Analytics

- Total Admissions
- Average Length of Stay
- Admissions by Month
- Hospital/Department Performance.

Home

Transform

Add Column

View

Tools

Help

Close & Apply

New Source

Recent Sources

Enter Data

Data source settings

Manage Parameters

Refresh Preview

Properties

Advanced Editor

Choose Columns

Remove Columns

Keep Rows

Remove Rows

Sort

Split Column

Group By

Use First Row as Headers

Replace Values

Data Type: Text

Merge Queries

Append Queries

Combine Files

Combine

Text Analytics

Vision

Azure Machine Learning

AI Insights

Queries [1]

HealthcareData

Name

1.2 Age

Gender

Blood_Type

Medical_Condition

Date_of_Admission

Doctor

Valid 100%

Valid 100%

Valid 100%

Valid 100%

Valid 100%

Valid 100%

Valid 100%

Error 0%

Error 0%

Error 0%

Error 0%

Error 0%

Error 0%

Error 0%

Empty 0%

Empty 0%

Empty 0%

Empty 0%

Empty 0%

Empty 0%

Empty 0%

1	Bobby JacksOn	30	Male	B-	Cancer	31-01-2024	Matthew Smi
2	Leslie Teriry	62	Male	A+	Obesity	20-08-2019	Samantha Da
3	DaNey sMth	76	Female	A-	Obesity	22-09-2022	Tiffany Mitch
4	andrew wATes	28	Female	O+	Diabetes	18-11-2020	Kevin Wells
5	adriENNE beI	43	Female	AB+	Cancer	19-09-2022	Kathleen Han
6	EMILY JOHNsOn	36	Male	A+	Asthma	20-12-2023	Taylor Newto
7	edwARd EDwARds	21	Female	AB-	Diabetes	03-11-2020	Kelly Olson
8	ChrisTiNA MARTinez	20	Female	A-	Cancer	28-12-2021	Suzanne Thor
9	JASmiNe sduIaR	82	Male	AB+	Asthma	01-07-2020	Daniel Fergus
10	CHRISTopher BERG	58	Female	AB-	Cancer	23-05-2021	Heather Day
11	michEile daniELS	72	Male	O+	Cancer	19-04-2020	John Duncan
12	aallon MARTiNez	38	Female	A-	Hypertension	13-08-2023	Douglas May
13	connOR HANsEN	75	Female	A+	Diabetes	12-12-2019	Kenneth Fiet

Query Settings

PROPERTIES

Name

HealthcareData

All Properties

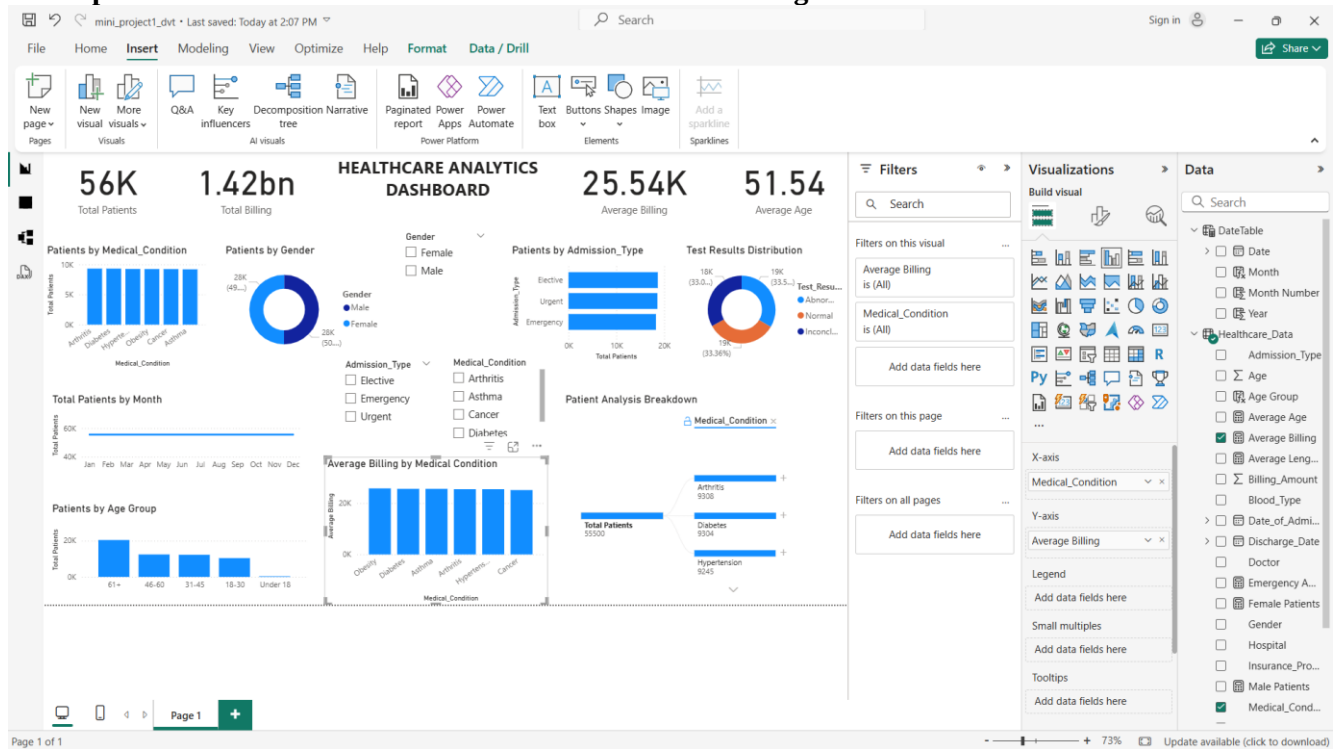
APPLIED STEPS

Source

Navigation

- >  DataTable
- >  Healthcare_Data ...
 - ☐ Admission_Type
 - ☐ Σ Age
 - ☐  Age Group
 - ☐  Average Age
 - ☐  Average Billing
 - ☐  Average Leng...
 - ☐ Σ Billing_Amount
 - ☐ Blood_Type
 - > ☐  Date_of_Admi...
 - > ☐  Discharge_Date
 - ☐ Doctor
 - ☐  Emergency A...
 - ☐  Female Patients
 - ☐ Gender
 - ☐ Hospital
 - ☐ Insurance_Pro...
 - ☐  Male Patients
 - ☐ Medical_Cond...
 - ☐ Medication
 - ☐ Name
 - ☐ Σ Room_Number
 - ☐ Test_Results

Step 4: Power BI Final Executive Dashboard & Drill-Through



1. Executive Canvas:

- Place **Card Visuals** across the top: Total Patients, Total Billings, Average Age, Average Billing.

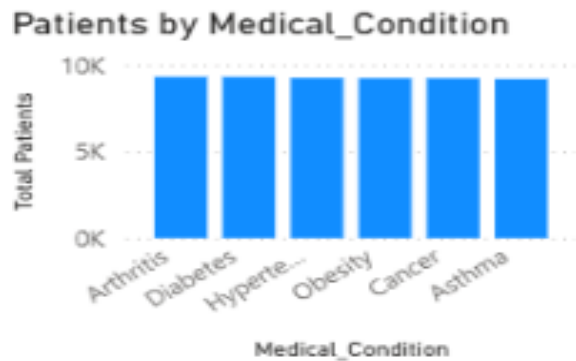


- Add a **Line Chart** mapped to Dim_Date[date] with hierarchy drill-down enabled (Year → Quarter → Month).

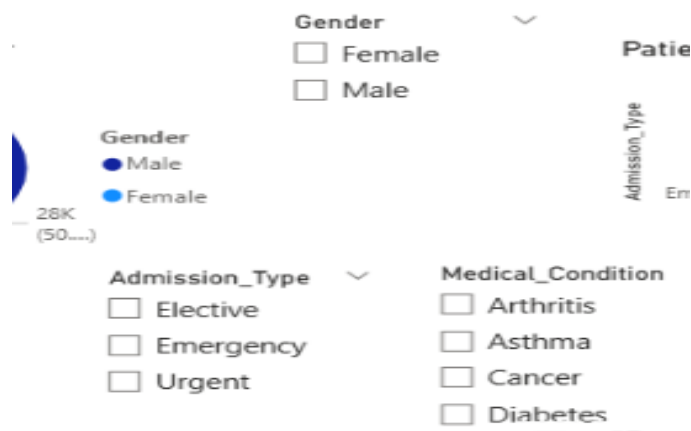
Total Patients by Month



- Add a **Clustered Bar Chart** for Medical_Condition

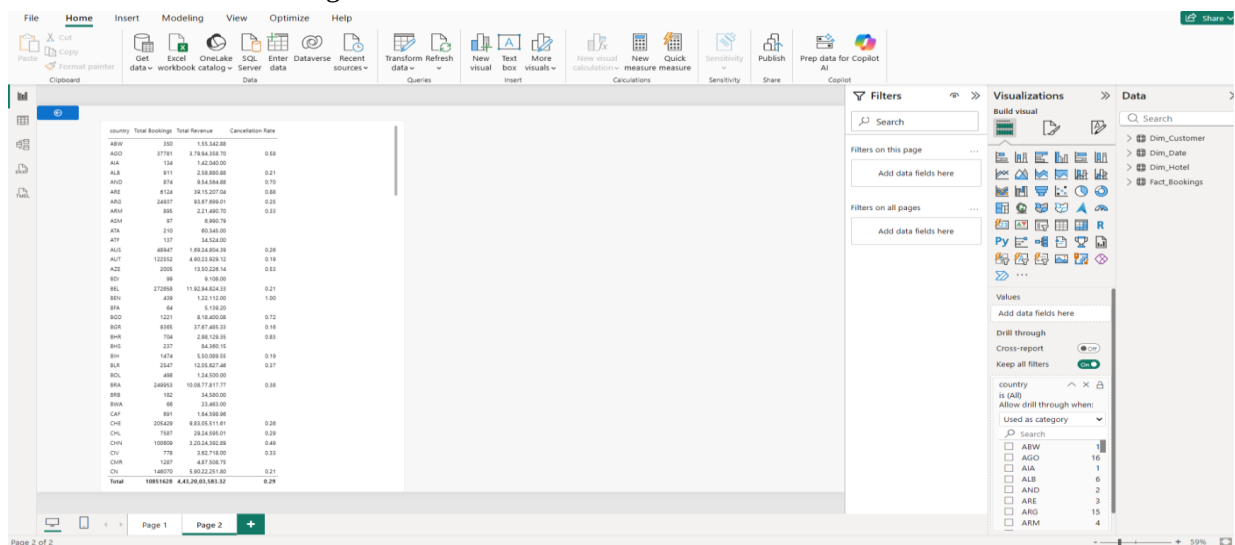


- Add **Slicers** for Gender, Admission type, Medical Condition.



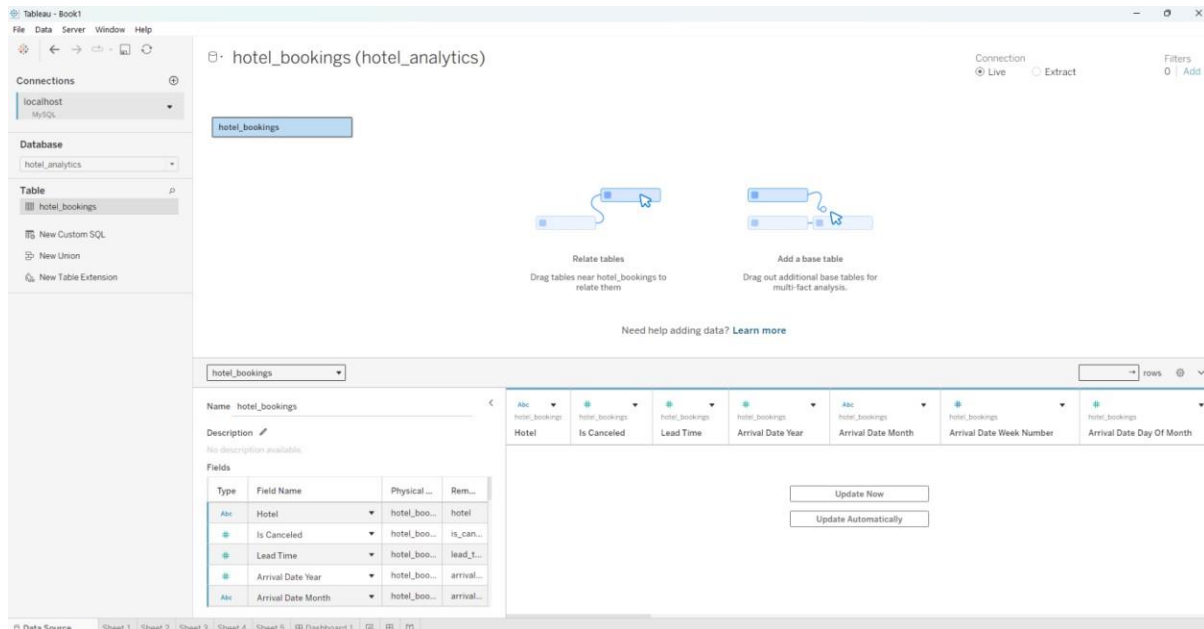
2. Country Details Drill-Through Page:

- Create a second page named Country Details.
- Drag Dim_Customer[country] into the **Drill-through** field bucket.
- Add a detailed Matrix/Table visual showing country, Total Bilings, Total Revenue, and Average Rate



Step 6: Tableau Data Connection & Preparation

1. Open Tableau Desktop / Tableau Public.
2. Under **To a Server**, select **MySQL**
3. Enter Server (localhost), Database (healthcare_analytics), and user credentials to pull the cleaned dataset onto the canvas.



- **Select Metric Parameter:** String list containing "Total Revenue", "Total Billings"

Edit Parameter [Select Metric]

Name
Select Metric

Properties
Data type: String
Display format: Total Revenue

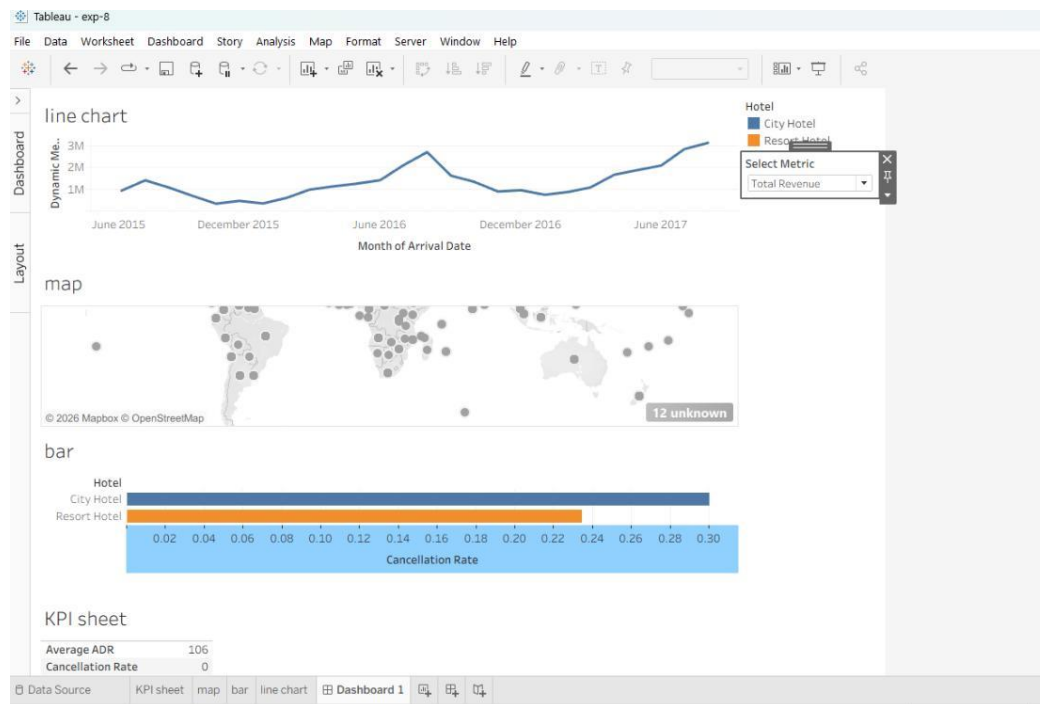
Current value: Total Revenue
Value when workbook opens: Current value

Allowable values
☐ All ☒ List ☐ Range

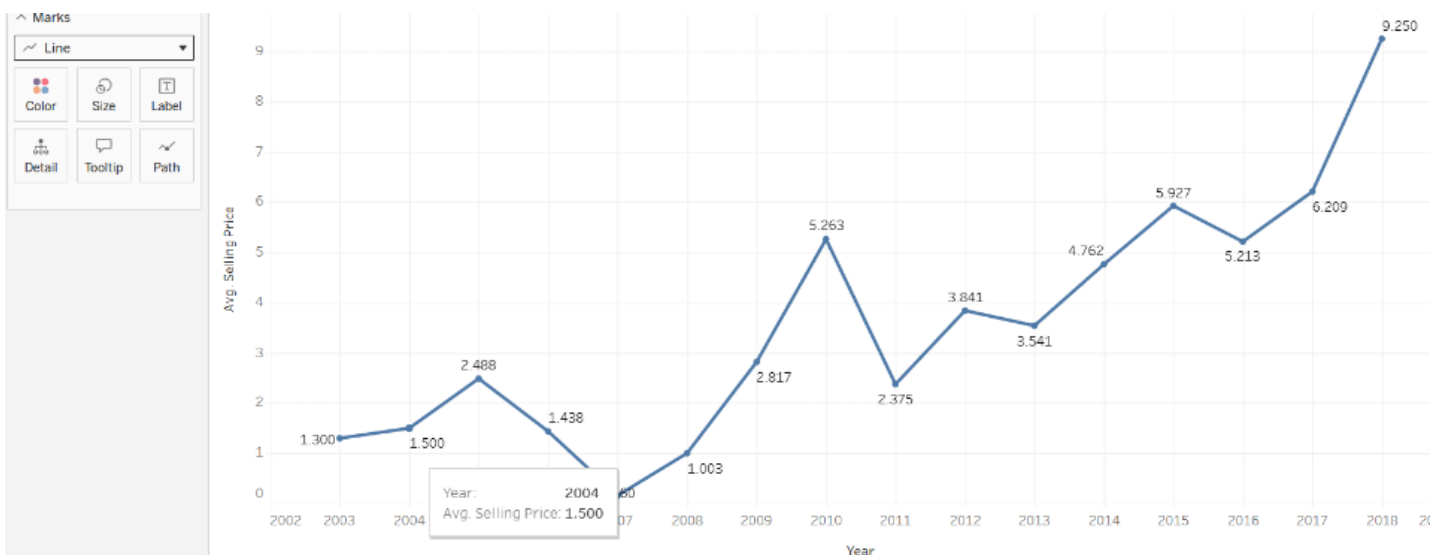
Value	Display As
Total Revenue	Total Revenue
Total Bookings	Total Bookings
Cancellation Rate	Cancellation Rate
Click to add	

☒ Fixed ☐ When workbook opens
Add values from
Remove Selected
Cancel OK

Step 7: Tableau Companion Dashboard & Interactive Story



1. **Worksheets:** Build **KPI Cards**, **Revenue Trend Line Chart** (using Dynamic Measure), **Hotel Cancellation Bar Chart**, and **Country Map**.

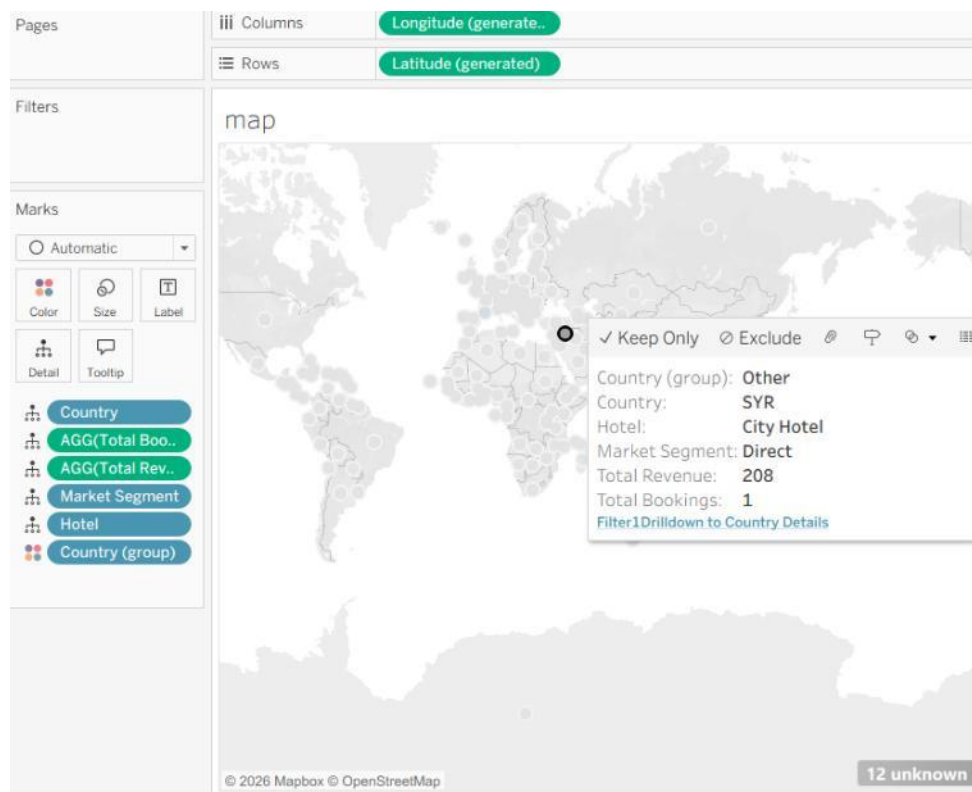


2. Global Filters & Actions:

- Drag Hotel, Deposit Type, and Arrival Date to Filters → set to **Apply to Worksheets** → **All Using This Data Source**.
- Drag sheets onto a new Dashboard tab and click the **Use as Filter** funnel icon on the Country Map visual.



3. Drill-down map:



Observed Insights:

- **Patient Volume & Healthcare Utilization:** The dataset captures overall patient records and hospital visits, providing insights into healthcare demand and utilization.
- **Patient Demographics:** Patient distribution varies across age groups and gender, highlighting key demographic segments requiring healthcare services.
- **Disease & Diagnosis Trends:** Common diseases and diagnoses account for a major share of patient visits, helping identify prevalent health conditions.
- **Treatment & Hospital Stay:** Length of stay and treatment patterns vary by diagnosis, indicating differences in healthcare resource utilization.
- **Healthcare Cost Analysis:** Medical expenses and billing amounts increase with complex treatments and longer hospital stays, highlighting major cost drivers.

RESULT:

Ex.no:9	MINI PROJECT – FINANCE& BANKING ANALYTICS

Domain and Problem Statement:

Domain: Banking Analytics & Financial Fraud Detection

Problem Statement: Financial institutions face capital loss and operational risk due to undetected transaction fraud, inefficient channel utilization, and poor visibility into state-level liquidity distribution. This project addresses these challenges by transforming raw transaction records into structured business intelligence across Power BI and Tableau to evaluate financial performance, geographic risk distribution, and fraud exposure across banking channels.

Dataset Description:

Dataset Name: Indian Banking Transactions Dataset

Granularity: Each row represents a single banking transaction.

Timeframe Covered: 2019 to 2024.

Key Entities & Attributes:

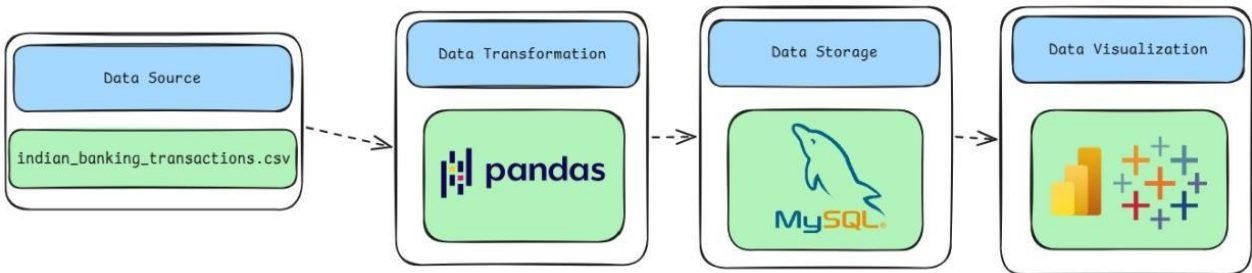
- Demographics & Account:** customer_id, state, credit_score, has_loan, loan_type, kyc_status, account_type, channel.
- Transaction Details:** transaction_id, transaction_date, transaction_time, transaction_hour, transaction_type, transaction_direction, merchant_category, transaction_status.
- Financials & Risk:** transaction_amount, account_balance, emi_amount, is_fraud.

Data Source:

Primary Source: Kaggle Open Datasets (Indian Banking Transactions).

Storage Infrastructure: Cleaned transactional records hosted within a local MySQL database instance (banking_analytics).

Architecture:



Program:

1. Python – Data Cleaning & Transformation

```
import pandas as pd

from sqlalchemy import create_engine, text

# Load dataset
df = pd.read_csv("indian_banking_transactions.csv")
bank = df.copy()

# Handle missing values
bank["customer_id"] = bank["customer_id"].fillna("Unknown")
bank["transaction_type"] = bank["transaction_type"].fillna("Unknown")
bank["account_type"] = bank["account_type"].fillna("Unknown")
bank["merchant_category"] = bank["merchant_category"].fillna("Unknown")
bank["transaction_direction"] = bank["transaction_direction"].fillna("Unknown")
bank["transaction_amount"] = bank["transaction_amount"].fillna(0)
bank["account_balance"] = bank["account_balance"].fillna(0)

# Remove duplicates
bank.drop_duplicates(inplace=True)

# Convert date column
bank["transaction_date"] = pd.to_datetime(
    bank["transaction_date"], errors="coerce"
)

# Standardize column names
bank.columns = (
    bank.columns.str.lower().str.strip().str.replace(" ", "_")
)

print("Original Shape:", df.shape)
print("Cleaned Shape:", bank.shape)

bank.head()
```

2. MySQL – Database and Data Loading

MySQL Command Prompt:

```
net start MySQL80
```

```
mysql -u root -p
```

MySQL:

```
CREATE DATABASE IF NOT EXISTS hotel_analytics;
```

Python/Jupyter:

```
# Database Server Credentials
```

```
username = "root"
```

```
password = "YOUR_MYSQL_PASSWORD"
```

```
host = "localhost"
```

```
port = 3306
```

```
# Create Database
```

```
server_engine = create_engine(
```

```
    f"mysql+pymysql://{username}:{password}@{host}:{port}"
```

```
)
```

```
with server_engine.connect() as connection:
```

```
    connection.execute(
```

```
        text("CREATE DATABASE IF NOT EXISTS banking_analytics")
```

```
)
```

```
# Connect to target DB and export table
```

```
db_engine = create_engine(
```

```
    f"mysql+pymysql://{username}:{password}@{host}:{port}/banking_analytics"
```

```
)
```

```
bank.to_sql(
```

```
    "banking_transactions",
```

```
    con=db_engine,
```

```
    if_exists="replace",
```

```
    index=False,
```


Procedure:

- Load raw transactional dataset using `pd.read_csv()`.
- Validate non-null constraints across mandatory fields (`transaction_id`, `customer_id`, `transaction_amount`).
- Format temporal values and export cleaned records into MySQL (`banking_analytics.banking_transactions`) via SQLAlchemy.

```
[2]: import pandas as pd
```

```
[1]: import kagglehub

# Download latest version
path = kagglehub.dataset_download("belbino/indian-banking-transactions-20192024")

print("Path to dataset files:", path)
```

C:\Users\santh\AppData\Local\Programs\Python\Python312\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
Downloading to C:\Users\santh\.cache\kagglehub\datasets\belbino\indian-banking-transactions-20192024\2.archive...
100% ██████████ 17.4M/17.4M [00:02<00:00, 6.59MB/s]
Extracting files...

Path to dataset files: C:\Users\santh\.cache\kagglehub\datasets\belbino\indian-banking-transactions-20192024\versions\2

```
[5]: df = pd.read_csv("indian_banking_transactions.csv")

print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

Rows: 550000
Columns: 20

```
[6]: df.head()
```

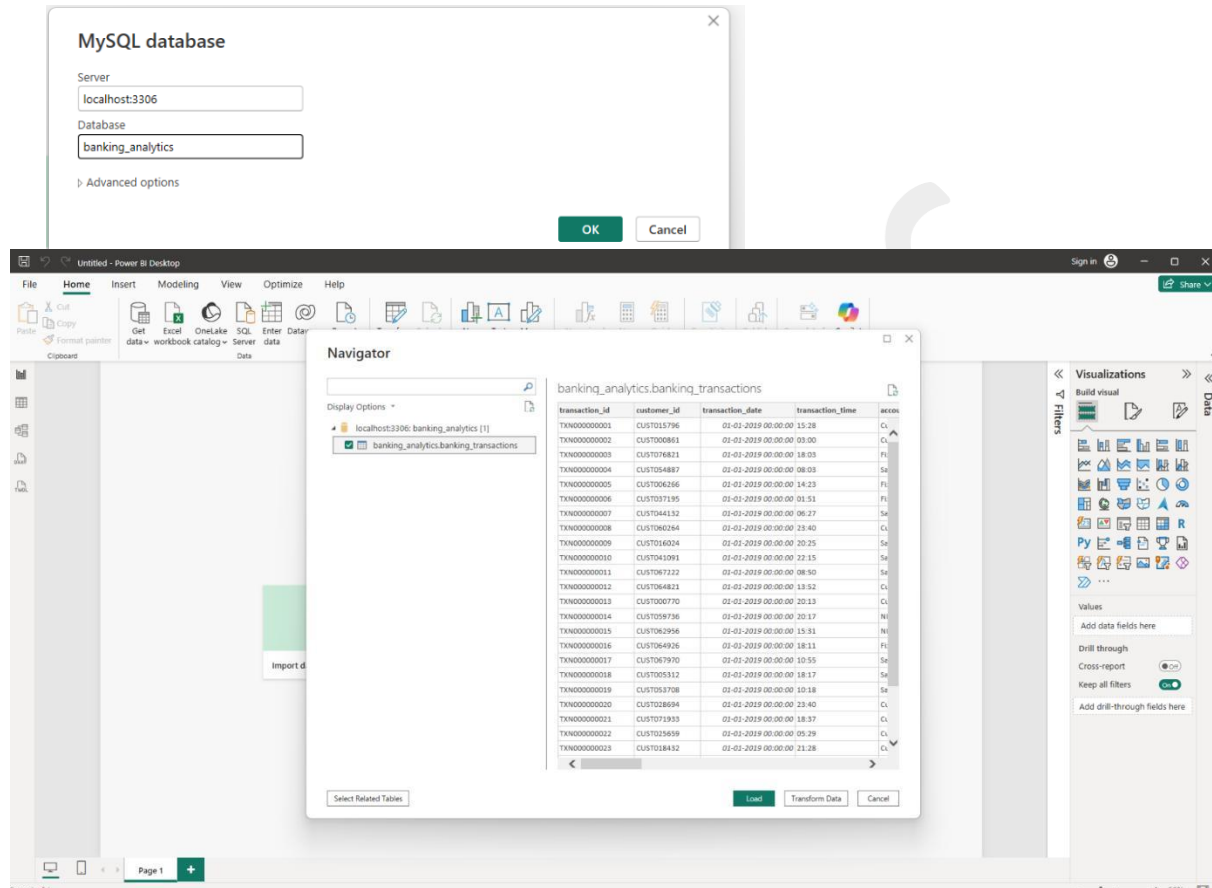
```
[6]:
```

	transaction_id	customer_id	transaction_date	transaction_time	account_type	transaction_type	transaction_amount	transaction_direction	account_balance	merchant
0	TXN000000001	CUST015796	2019-01-01	15:28	Current	UPI	1820.17	Debit	609365.31	Food
1	TXN000000002	CUST000861	2019-01-01	03:00	Current	UPI	392.67	Credit	14451.14	
2	TXN000000003	CUST076821	2019-01-01	18:03	Fixed Deposit	POS	1255.97	Debit	47621.87	
3	TXN000000004	CUST054887	2019-01-01	08:03	Savings	UPI	2580.68	Debit	34467.85	
4	TXN000000005	CUST006266	2019-01-01	14:23	Fixed Deposit	UPI	2573.80	Debit	26617.40	

```
[7]: df.info()
```

Step 2: SQL Database Connection & Data Ingestion

1. Open Power BI Desktop → Click **Get Data** → Select **MySQL Database**.
2. Enter Server (localhost:3306) and Database Name (banking_analytics).
3. Authenticate with root credentials, select banking_transactions, and click **Transform Data** to launch Power Query.



Step 3: Star Schema Data Modeling (Power BI)

1. Table Splitting in Power Query:

Duplicate raw query banking_transactions into 3 separate entities: Fact_Transactions, Dim_Customer, and Dim_Account.

Dim_Customer: Retain customer_id, state, credit_score, has_loan, loan_type, kyc_status. Remove duplicates on customer_id (79,916 unique rows).

Dim_Account: Retain account_type and channel. Remove duplicates and add an Index Column named **AccountKey**.

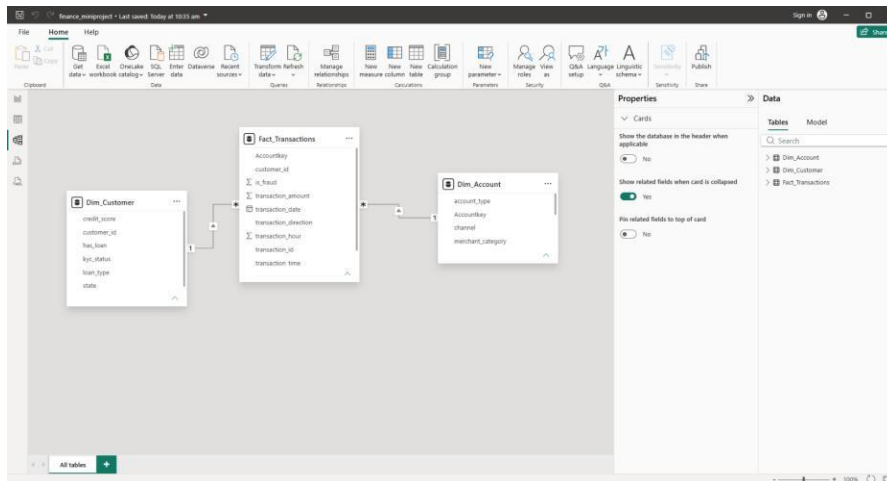
Fact_Transactions: Merge with Dim_Account on account_type and channel to pull in AccountKey. Then remove redundant text columns (state, credit_score, has_loan, loan_type, kyc_status, account_type, channel).

2. Establishing Relationships in Model View:

Connect Dim_Customer[customer_id] → Fact_Transactions[customer_id] (1:*).

Connect Dim_Account[AccountKey] → Fact_Transactions[AccountKey] (1:*).

Hide foreign key columns inside Fact_Transactions to enforce clean report pane structure.



Step 4: Core Calculations & Measures (DAX)

Power BI Measures (DAX)

Create a dedicated _Measures table and add the following DAX calculations:

- **Total Transactions:**

```
1 Total Transactions = COUNTROWS(Fact_Transactions)
```

- **Total Transaction Amount:**

```
1 Total Transaction Amount = SUM(Fact_Transactions[transaction_amount])
```

- **Average Transaction:**

```
1 Average Transaction = AVERAGE(Fact_Transactions[transaction_amount])
```

- **Total Customers:**

```
1 Total Customers = DISTINCTCOUNT(Fact_Transactions[customer_id])
```

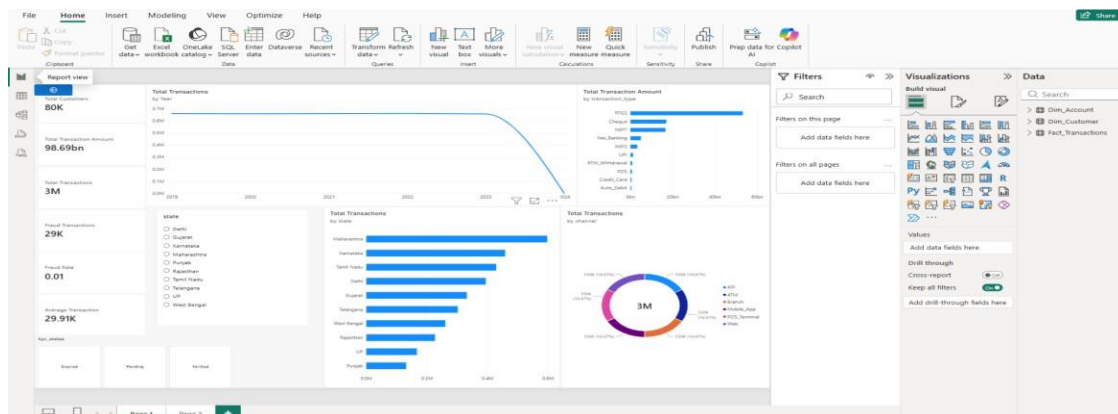
- **Fraud Transactions:**

```
1 Fraud Transactions = CALCULATE(COUNTROWS(Fact_Transactions), Fact_Transactions[is_fraud] = 1)
```

- **Fraud Rate (%):**

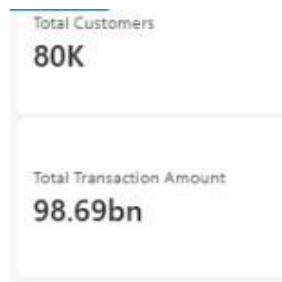
```
Fraud Rate = DIVIDE([Fraud Transactions], [Total Transactions], 0)
```

Step 5: Power BI Final Executive Dashboard & Drill-Through

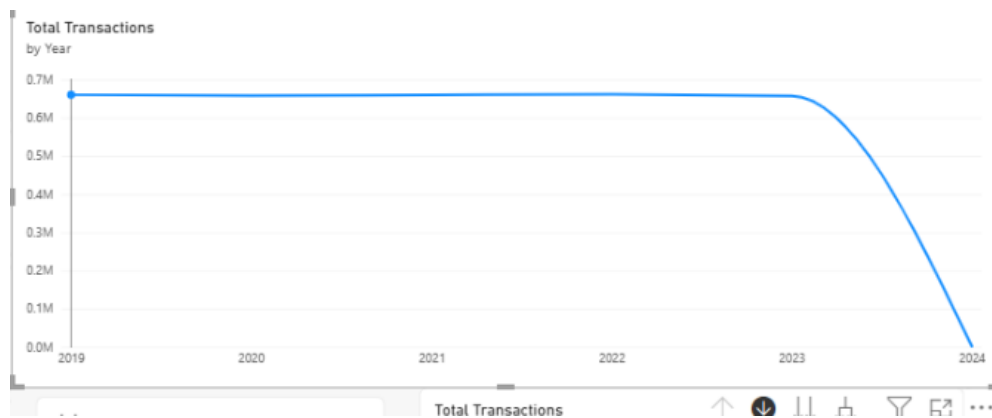


1. Executive Canvas:

- Place **Card Visuals** across the top: Total Customers, Total Transaction Amount, Total Transactions, Fraud Transactions, Fraud Rate, Average Transaction

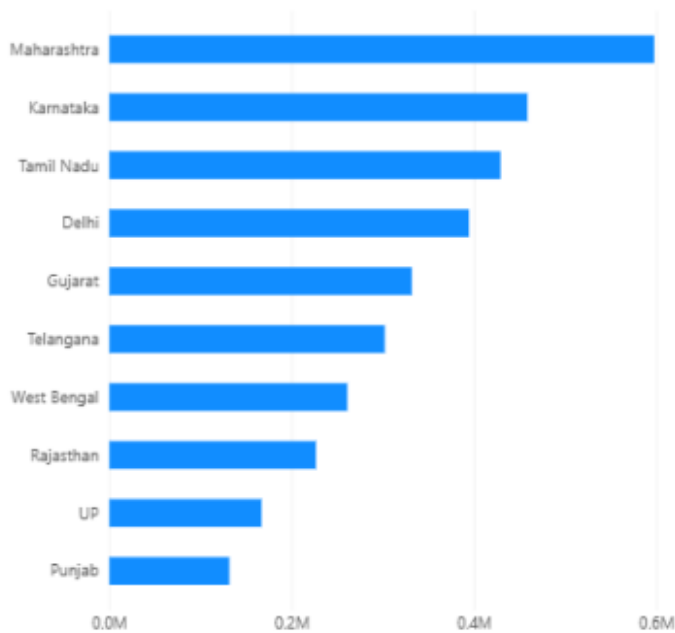


- Add a **Line Chart** mapped to Total Transactions by Year.

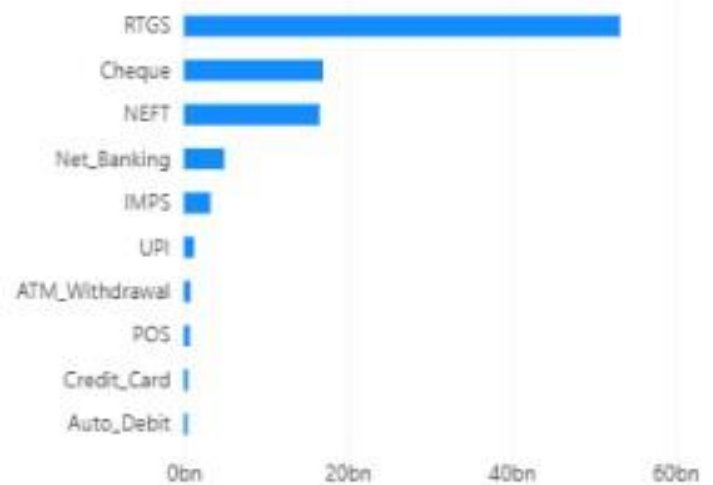


- Add a **Clustered Bar Chart** for for Total Transactions by state

Total Transactions
by state

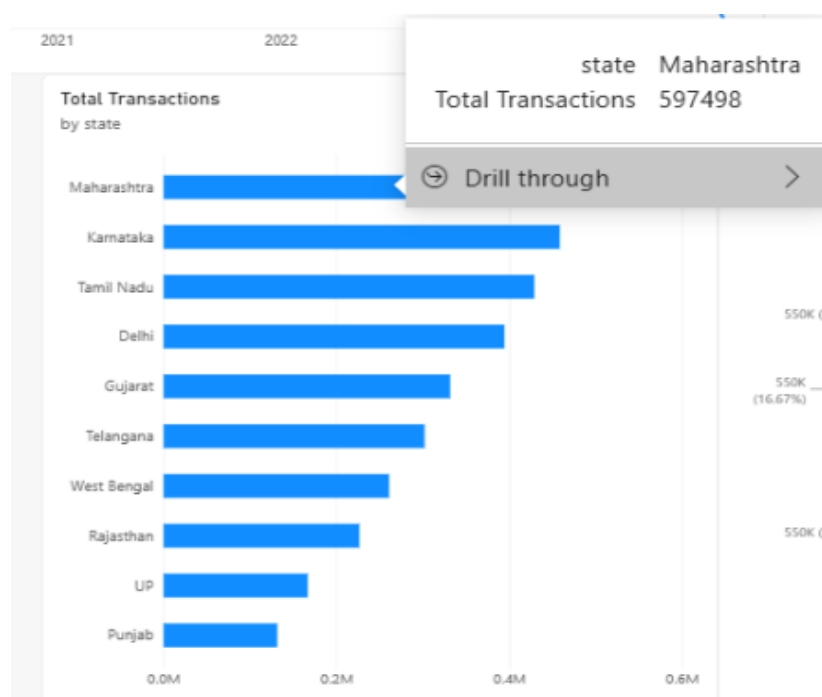
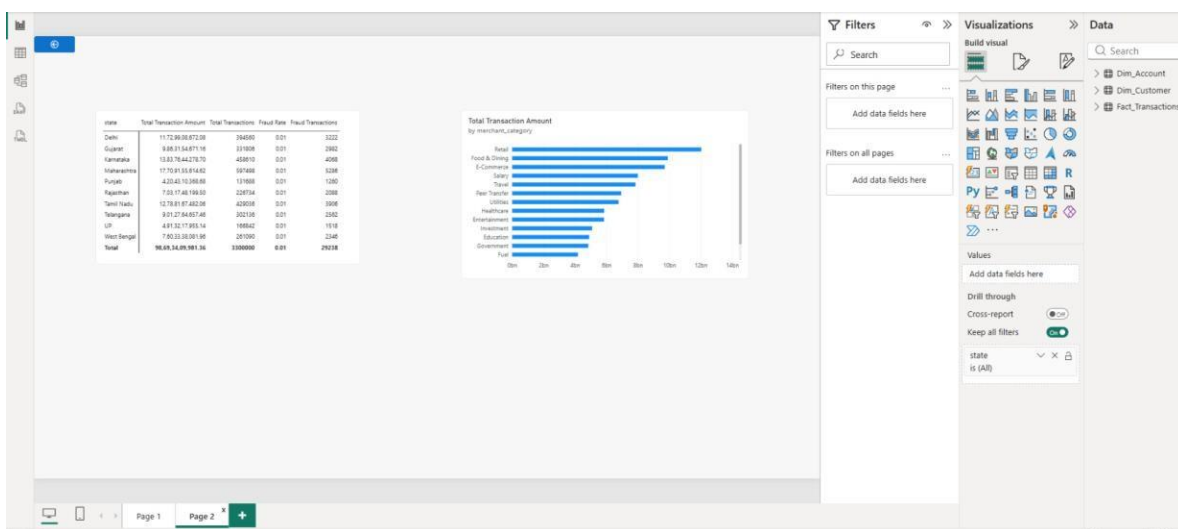


Total Transaction Amount
by transaction_type



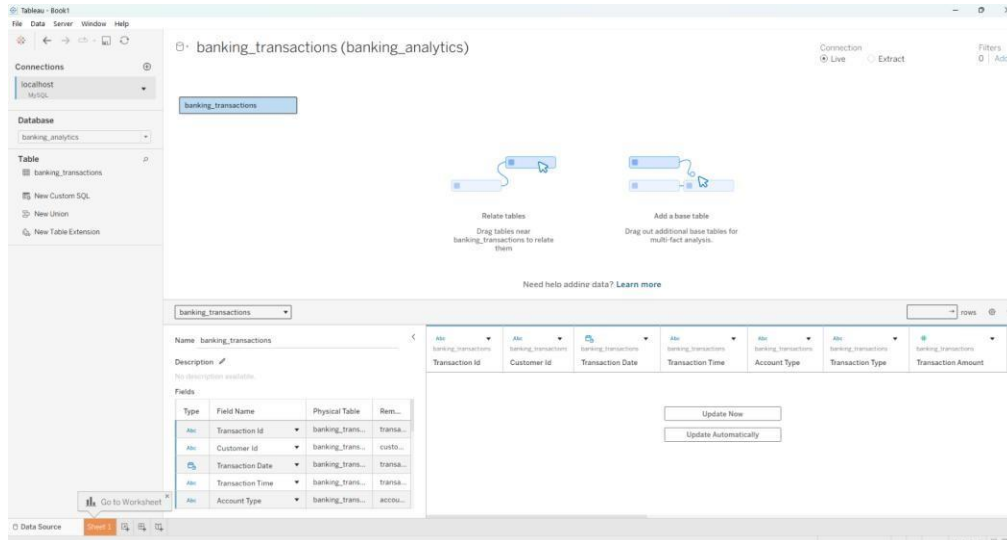
2. State Details Drill-Through Page:

- Create a second page named Country Details.
- Drag Dim_Customer[country] into the **Drill-through** field bucket.
- Add a detailed Matrix/Table visual showing country, Total Bookings, Total Revenue, and Cancellation Rate
- Create a second page named Page 2.
- Drag Dim_Customer[state] into the Drill-through field bucket.
- Add a detailed Matrix/Table visual showing state, Total Transaction Amount, Total Transactions, Fraud Rate, and Fraud Transactions.
- Add a Bar Chart for Total Transaction Amount by merchant_category.



Step 6: Tableau Data Connection & Preparation

1. Open Tableau Desktop / Tableau Public.
2. Under **To a Server**, select **MySQL**.
3. Enter Server (localhost), Database (banking_analytics), and user credentials to pull the cleaned dataset onto the canvas.



1. Go to any sheet and build calculated fields:

- **Total Transaction amount:**

Total Transaction amount banking_transactions (banking_analytics)

```
SUM([Transaction Amount])
```

- **Total transactions:**

Total transactions banking_transactions (banking_analytics)

```
COUNT([Transaction Id])
```

- **Fraud rate:**

Fraud rate banking_transactions (banking_analytics)

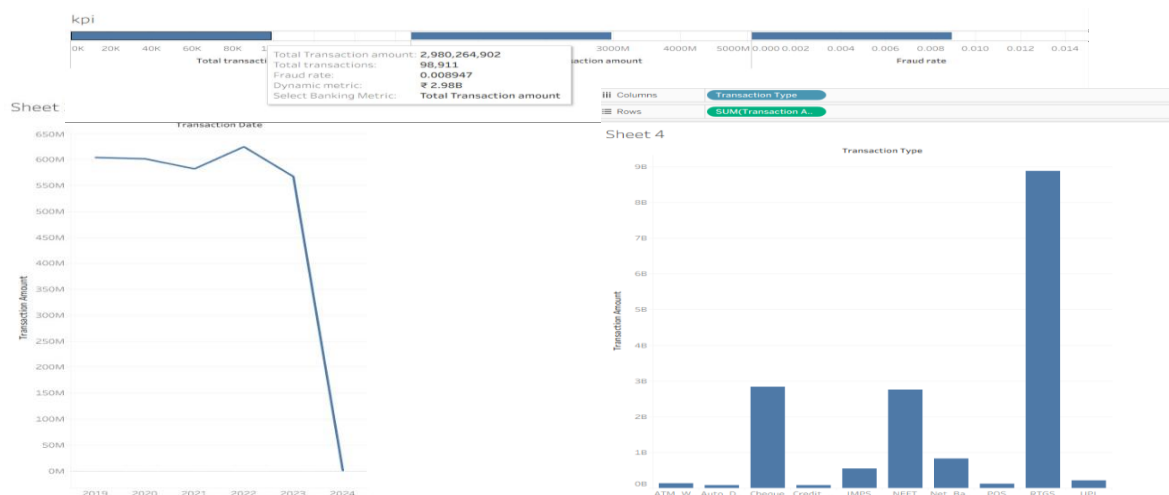
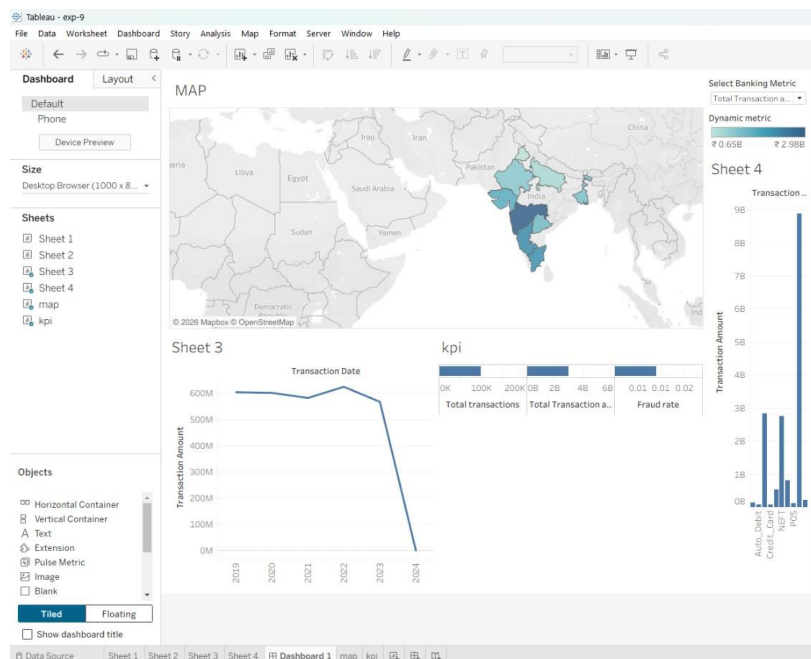
```
SUM(INT([Is Fraud])) / COUNT([Transaction Id])
```

- **Dynamic Metric Formula:**

Dynamic metric banking_transactions (banking_analytics)

```
CASE [Select Banking Metric]
  WHEN 'Total Transaction amount' THEN [Total Transaction amount]
  WHEN 'Total transactions' THEN [Total transactions]
  WHEN 'Fraud rate' THEN [Fraud rate]
END
```

Step 7: Tableau Companion Dashboard & Interactive Story



- Worksheets:** Build **KPI sheet**, **Hourly Trend Line Chart** (using Dynamic Metric), Channel Breakdown **Bar Chart**, and **State Map**.
- Global Filters & Actions:**
 - Add Parameter control for Select Banking Metric to the dashboard.
 - Drag sheets onto a new Dashboard tab and click the **Use as Filter** funnel icon on the State Map visual.

Edit Filter Action

Name

State Map Drill-Through

Insert

Source Sheets

Dashboard 1

kpi

map

Sheet 3

Sheet 4

Run action on

Hover

Select

Menu

Single-select only

Target Sheets

Dashboard 1

kpi

map

Sheet 3

Sheet 4

Clearing the selection will

Keep filtered values

Show all values

Exclude all values

Filter

All fields

Selected fields

Source Field

Target Data Source

Target Field

Click to add

Remove

Cancel

OK

RESULT: